Lecture 20: Classes (Chapter 13)

유준수 (Joon Soo Yoo, Ph.D.)

2026년 4월 16일

방산첨단보안연구실 (DAS Lab)
전북대학교 첨단방위산업학과

13.1 Inheritance Basics

코드의 가업 잇기: 상속의 개념과 구조

상속(Inheritance)이란?

이미 만들어진 클래스(부모)의 모든 속성과 메서드를 새로운 클래스(자식)가 그대로 물려받는 것입니다.

- **중복 제거:** 똑같은 코드를 여러 번 쓸 필요가 없습니다.
- **확장성:** 부모의 기능을 기반으로 새로운 기능만 추가하면 됩니다.
- **유지보수:** 부모 클래스만 잘 수정하면 이를 상속받은 모든 자식 클래스에 반영됩니다.

설계의 핵심: is-a vs has-a

클래스 간의 관계를 정확히 정의해야 올바른 상속을 할 수 있습니다.

is-a (상속 관계)

"자식은 부모의 한 종류이다."
전사(`Warrior`) **is-a** 캐릭터(`Character`)
→ 상속을 사용합니다.

has-a (포함 관계)

"객체가 다른 객체를 가지고 있다."
전사 **has-a** 칼(`Sword`)
→ 상속이 아닌 '변수'로 포함합니다.

상속 vs 포함: 헷갈릴 때 던지는 질문

1. 상속(is-a): "A는 B이다" (분류학적 관계)

하위 클래스가 상위 클래스의 **개념적 범주** 안에 완전히 포함될 때 사용합니다.

- **자연스러운 표현:** "참새는 새다", "노트북은 컴퓨터다"

2. 포함(has-a): "A는 B를 가지고 있다" (소유/구성 관계)

한 객체가 다른 객체를 **부품이나 도구**로 사용할 때 사용합니다.

- **자연스러운 표현:** "자동차는 엔진을 가지고 있다", "학생은 가방을 메고 있다"

설계 가이드

전사 `is a` 칼? (X) → 전사가 칼일 수는 없으니 상속 금지!
전사 `has a` 칼? (O) → 전사 클래스 안에 칼 변수를 만드세요.

파이썬 상속 문법

클래스를 정의할 때 괄호 안에 부모 클래스의 이름을 적어줍니다.

상속 기본 구조

```
class Parent:   # 부모 클래스 (Superclass)
    def hello(self): print("안녕하세요")

class Child(Parent):   # 자식 클래스 (Subclass)
    pass

c = Child()
c.hello()   # 부모의 메서드를 내 것처럼 사용!
```

함수(Function) vs 메서드(Method)

둘 다 특정 기능을 수행하는 코드 블록이지만, "누구에게 속해 있는가"가 핵심입니다.

함수 (Function)

독립적으로 존재하며 이름을 통해 직접 호출합니다.

- 예: `print()`, `len()`, `sum()`
- 특정 객체의 상태를 직접 참조하지 않습니다.

```
result = sum([1, 2, 3])
```

메서드 (Method)

클래스 내부에 정의되어 객체를 통해 호출합니다.

- 예: `my_list.append()`, `a1.attack()`
- `self`를 통해 객체의 데이터(속성)에 접근합니다.

```
a1.attack(a2)
```

결론: 메서드는 "객체가 수행하는 함수"라고 이해하면 된단다!

누가 무엇을 사용할 수 있나?

상속은 일방통행입니다. 자식은 부모의 것을 다 알지만, 부모는 자식의 전용 기능을 모릅니다.

접근 권한 정리

- **자식 인스턴스:** 부모 메서드 + 자식 메서드 **모두** 사용 가능
- **부모 인스턴스:** 부모 메서드만 사용 가능 (자식 기능 접근 불가)

용어	부모 클래스	자식 클래스
일반 용어	Superclass (슈퍼 클래스)	Subclass (서브 클래스)
기초 용어	Base class (기초 클래스)	Derived class (파생 클래스)
가족 비유	Parent class (부모 클래스)	Child class (자식 클래스)

Task 1: RPG 캐릭터 전직 시스템 설계

캐릭터가 전직할 때마다 이전 단계의 기술을 그대로 간직하며 새로운 기술을 배우는 '상속의 흐름'을 구현해 봅시다.

Mission: 3단계 전직 구조 만들기

1. **GameCharacter (초보자):** `move()` 메서드 ("기본 이동을 합니다")
2. **Warrior (전사):** `GameCharacter`를 상속받고, `slash()` 메서드 ("칼을 휘두릅니다")
3. **Berserker (광전사):** `Warrior`를 상속받고, `rage()` 메서드 ("폭주 상태가 됩니다!")
4. **실험:** `Berserker` 인스턴스를 하나 만들고, 초보자 기술(`move`), 전사 기술(`slash`), 광전사 기술(`rage`)을 모두 호출해 보세요.

13.2 Attribute Access

부모의 유산 관리: 속성 상속과 확장

자식 클래스의 기본값: 부모의 __init__ 빌려 쓰기

자식 클래스에서 __init__을 따로 정의하지 않으면, 파이썬은 자동으로 부모 클래스의 __init__을 호출합니다.

자동 상속 예시

```
class Character:
  def __init__(self):
    self.hp = 100

class Warrior(Character):
  pass   # 자신의 __init__이 없으면 부모의 것을 사용

p1 = Warrior()
print(p1.hp)   # 결과: 100 (부모가 준 초기 체력)
```

새로운 속성 추가하기: 자식만의 개성 부여

부모로부터 물려받은 속성 외에, 자식 클래스만의 새로운 속성을 추가하고 싶을 때 `__init__`을 재정의합니다.

속성 확장의 원리

- **기존 속성:** 부모 클래스에서 정의된 변수들 (예: `hp`, `name`)
- **추가 속성:** 자식 클래스에서만 사용하는 변수 (예: `power`, `weapon`)

중요: 오버라이딩(Overriding)

자식 클래스에서 `__init__`을 직접 쓰면, 부모의 `__init__`은 무시됩니다. 따라서 부모가 가진 기본 속성들도 자식의 `__init__` 안에 다시 정의해주어야 한단다.

예시: 계좌 정보 상속과 확장

부모: Account (일반 계좌)

```
class Account:
    def __init__(self, owner):
        self.owner = owner
        self.balance = 0
```

자식: SavingsAccount (적금)

```
class SavingsAccount(Account):
    def __init__(self, owner, rate):
        # 부모의 속성을 직접 다시 정의
        self.owner = owner
        self.balance = 0
        # 적금 계좌만의 새로운 속성
        self.interest_rate = rate
```

실험 및 검증

```
acc = SavingsAccount("최용준", 0.05)
print(acc.owner, acc.balance, acc.interest_rate)
```

→ **결과: 최용준 0 0.05 (정상 확인!)**

Task 2: 광전사(Berserker)의 탄생

기본 전사(`Warrior`) 클래스를 상속받아, '분노(Rage)' 게이지를 가진 광전사를 만들어 봅시다.

Mission: 속성 상속과 확장 구현

1. **Warrior:** `__init__`에서 `name="Warrior"`와 `hp=100`을 초기화합니다.
2. **Berserker:** `Warrior`를 상속받습니다.
3. **속성 추가:** `Berserker`의 `__init__`에서 부모의 `name, hp`를 설정하고, 추가로 `rage=50`을 초기화하세요.
4. **출력:** `display()` 메서드를 만들어 "이름: name, 체력: hp, 분노: rage"를 출력하세요.

13.3 Methods & Polymorphism

상속의 완성: 오버라이딩과 다형성

메서드 오버라이딩 (Method Overriding)

부모에게 물려받은 메서드가 마음에 들지 않거나, 자식만의 특별한 기능이 필요할 때 **이름은 그대로 두고 내용만 새로 정의**하는 것입니다.

부모: 일반 공격

```
def attack(self):
  print("일반 공격!")
```

자식: 광전사 공격 (Overridden)

```
def attack(self):
  print("!!!광폭 공격!!!")
```

핵심 규칙

메서드 이름이 부모와 똑같아야 합니다. 자식 인스턴스에서 호출하면 **자식의 오버라이딩된 메서드**가 우선권을 가집니다.

마법의 단어: super()

부모의 코드를 복사해서 붙여넣는 수고를 덜어줍니다. 자식 클래스에서 **부모 클래스의 메서드를 호출**하고 싶을 때 사용합니다.

super() 활용: 계좌 시스템

```
class SavingsAccount(Account):
    def __init__(self, owner, rate):
        super().__init__(owner)  # 부모의 __init__을 대신 실행해줘!
        self.interest_rate = rate
```

- **효율성:** 부모가 이미 만든 코드를 재사용하여 중복을 줄입니다.
- **유지보수:** 부모의 로직이 바뀌어도 자식 코드를 수정할 필요가 없습니다.

다형성(Polymorphism): 하나의 명령, 다른 결과

'Poly(여럿)' + 'Morph(형태)'의 합성어로, **동일한 이름의 메서드**가 객체에 따라 다르게 동작하는 성질입니다.

- **사용자 편의성:** 객체가 정확히 무엇인지 몰라도, 이름만 같으면 똑같은 방식으로 명령을 내릴 수 있습니다.
- **구현의 유연성:** 부모 클래스의 메서드를 자식들이 각자 자기 방식대로 **오버라이딩(Overriding)** 함으로써 실현됩니다.

비유: 자동차의 가속 페달

운전자는 '페달'만 밟으면 됩니다. 전기차는 전기 모터로, 가솔린차는 엔진 폭발로 가속하지만, 운전자에게는 똑같은 '가속' 인터페이스일 뿐입니다.

다형성의 진수: 통합 관리

다형성을 이용하면 서로 다른 클래스의 객체들을 하나의 리스트에 담아 **한 번에 제어**할 수 있습니다.

객체 지향적인 반복문

```
units = [Warrior("용준"), Mage("민석"), Archer("진성")]


for u in units:
    u.attack()  # u가 누구든 상관없이 'attack'이라는 이름으로 명령!
```

- **결과:** "칼을 휘두릅니다!", "마법을 씁니다!", "활을 씁니다!"가 각각 출력됩니다.
- **확장성:** 새로운 캐릭터 Healer가 추가되어도, 위 반복문 코드는 **단 한 줄도 고칠 필요가 없습니다.**

파이썬이 다형성을 다루는 법: Duck Typing

- **덕 타이핑(Duck Typing):** 상속 관계가 아니더라도 메서드 이름만 같다면 파이썬은 군말 없이 실행해 줍니다.
- **내장 함수의 다형성:** `len()` 함수는 문자열, 리스트, 딕셔너리 등 내부적으로 `__len__`을 가진 모든 객체에 작동합니다.
- **연산자 다형성:** + 기호는 숫자에서는 '더하기', 문자열에서는 '이어붙이기'가 됩니다.

Task 3: 전직 시스템 완성하기

`super()`와 오버라이딩을 사용하여 `Berserker`를 완성해 봅시다.

Mission: `super()`와 오버라이딩

1. **`__init__`:** `super()`를 사용하여 부모(`Warrior`)의 `name`을 초기화하세요.
2. **attack:** 부모의 `attack()`을 오버라이딩하여 "분노의 일격!"을 출력하세요.
3. **검증:** `b = Berserker("광전사 김")`을 만들고 `b.attack()`을 호출해 보세요.

Task 4: 드론 통합 이륙 시스템

기종이 다른 드론들을 하나의 명령어로 동시에 이륙시키는 시스템을 설계해 봅시다.

Mission: 다형성 실습

1. **BaseDrone:** `take_off()` 메서드 ("기본 이륙") 정의
2. **QuadCopter:** `take_off()` 오버라이딩 ("프로펠러를 돌려 수직 이륙")
3. **FixedWing:** `take_off()` 오버라이딩 ("활주로를 달려 이륙")
4. **실험:** 리스트에 두 드론 인스턴스를 넣고 `for` 문으로 `take_off()`를 한 번에 호출하세요.

Tip

수백 대의 드론을 조종할 때 기종마다 명령어가 다르다면 대혼란이 옵니다. **'이륙'**이라는 공통 인터페이스로 모든 기체를 제어하는 것, 그것이 다형성의 힘입니다.